FLAME Platform

Technical & Feature Documentation

For each feature: step-by-step usage with a screenshot, and the full implementation — frontend, API (request/response), database tables, and business rules — plus a typed database reference and an ERD.



1. Architecture Overview

Two-tier web application:

• Client — React 18 + Vite + Tailwind CSS (dev port 5173). react-router-dom routing; axios API client with a JWT interceptor; react-leaflet maps.

• Server — Node.js + Express REST API (port 4000). Prisma ORM over SQLite (swap provider to PostgreSQL for production). JWT auth (jsonwebtoken), bcrypt password hashing, Multer uploads (≤12MB PDF / ≤2MB other) served from /uploads.

Unified concept — Missions and MissionLogs are the same idea: creating a mission generates one MissionLog per visited school; the public dashboard reads these logs. School-level logs/tasks are the same MissionLog records created directly on a school.

Status model — Expected (not started) · Pending (in progress) · Completed. Dashboard date filters treat the end date as end-of-day so same-day entries always count.

Auth & roles — admin (full + the only one who manages users), content_editor, reviewer, form_manager. requireRole grants access if any of a user's roles is permitted.

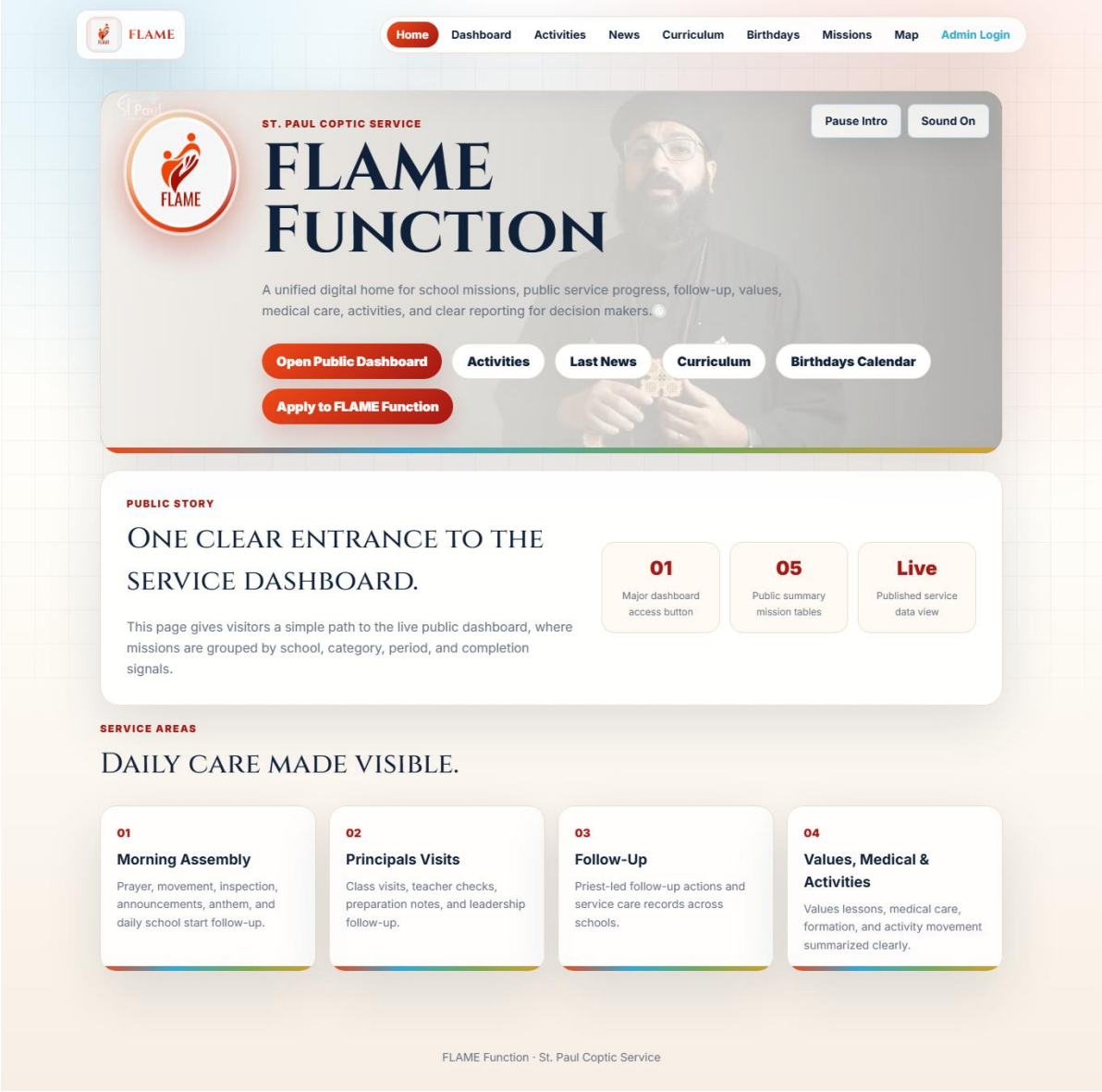
2. Features

2.1 Public Home & Video Hero

Landing page introducing the FLAME service with an intro video, animated hero, quick links, and an Apply call-to-action.

How to use it

1. Open the site root (/) in a browser.
2. The hero plays intro.mp4 with Pause / Sound controls.
3. Use the navbar to reach Dashboard, Missions, Activities, News, Curriculum, Birthdays, or Map.
4. Click 'Apply' to open the dynamic application form popup.
5. Fill the fields rendered from the admin-defined form and submit.



Business rules

- The Apply form is fully data-driven: fields come from FormField records ordered by `order` where `active = true`.
- A submission stores all answers as a JSON blob in Application.values.

Frontend

React page pages/public/Home.jsx composes components/VideoHero.jsx (HTML5 video + play/mute state) and components/ApplyModal.jsx. Styling: Tailwind utility classes + CSS design tokens in index.css (Cinzel/Inter fonts, glass-morphism cards, fade-up keyframe animations). Routing by react-router-dom.

API

GET `/api/form-fields`

List active form fields for the Apply popup.

Response

```
[ { id, label, key, type, options, required, order, active } ]
```

POST `/api/applications`

Submit the Apply form.

Request

```
{ "values": { "name": "...", "country": "...", "role": "..." } }
```

Response

```
201 { id, values, submittedAt }
```

Database tables used

FormField, Application

2.2 Public Missions Dashboard

Live public story of the service: KPIs, Schools×Missions summary, per-school drill-down of tasks performed vs. not performed, and category totals — all driven by the date range and filters.

How to use it

6. Open /dashboard.
7. Set Period or From/To dates — the whole dashboard reloads automatically.
8. Filter by Log Type, School, or Status (Completed / Expected / Pending).
9. Read the KPI strip: Total Missions, Active Schools, Completed, Expected, Pending.
10. Open any school to drill into task records grouped by log type.

Business rules

- The `to` date is treated as end-of-day (23:59:59.999) so same-day logs are always included.
- Total Missions counts DISTINCT real missions (by the mission id tagged in a log's notes); a mission visiting N schools counts once but shows activity in all N schools.
- A unit is Completed if its tasks are all performed or status = completed; Expected if status = expected; otherwise Pending.

Frontend

pages/public/Dashboard.jsx — a useEffect keyed on every filter (from/to/period/type/school/status) reloads the data; KpiCard.jsx cards; nested <details> drill-down per school and per log type.

API

GET `/api/dashboard/summary`

Aggregated KPIs + per-school + per-category.

Request

```
query: from, to, schoolId?, typeId?, status?
```

Response

```
{ range, kpis:{ totalMissions, activeSchools, completionSignal, completed,
pending, expected }, bySchool:[...], byCategory:[...] }
```

GET `/api/logs`

Raw logs for the detailed story.

Request

```
query: from, to, schoolId?, typeId?, status?
```

Response

```
[ { id, logTypeId, schoolId, logDate, status, notes, tasks:[...] } ]
```

Database tables used

MissionLog, TaskRecord, LogType, School

2.3 🚑 Missions (Admin): Status, Dates & Editing

Create missions (Medical / Follow-ups / Activities / Values / Formation / Other) with mandatory status, start/end dates and at least one school. Each mission auto-generates dashboard logs and is fully editable.

How to use it

11. Admin → Missions.
12. Enter a title; pick type and status (Expected / Pending / Completed).
13. Set mandatory start and end dates; select at least one school.
14. Add countries (per-country dates), people and medical team — availability is shown.
15. Create — one dashboard log per visited school is generated concurrently.
16. Use Edit to change status/dates/summary; members become unavailable while a mission is Pending and free again when Completed or deleted.

Business rules

- Server-side validation: title, status, startDate, endDate and ≥1 school are required; endDate must be ≥ startDate (HTTP 400 otherwise).
- Generated MissionLog.logDate uses the mission startDate; log status mirrors the mission status.
- Mission type maps to a dashboard LogType (Medical→medical_log, Follow-ups→follow_up, Activities→activities, Values→values_class, Formation→formation); 'Other' creates no dashboard log.
- Deleting a mission cascades to its MissionCountry/School/Person/Medical rows and removes its generated logs + task records.
- The list endpoint returns country names + counts only (not the full nested graph) to keep it fast under load.

Frontend

components/admin/MissionsAdmin.jsx — create form (status dropdown, date pickers, country rows, school/person/medic pickers with availability badges) + inline Edit panel.

API

`GET /api/missions`

List (type filter optional). Returns counts + country names.

Request

```
query: type?
```

Response

```
[ { id, title, type, status, startDate, endDate, summary, anonymousCount,
countries:[{country:{name}}], schools:[len], people:[len], medicalTeam:[len] } ]
```

POST `/api/missions`

Create a mission (+ generate logs).

Request

```
{ title, type, status, startDate, endDate, summary, numPeople, durationDays,
anonymousCount, countries:[{countryId,startDate,endDate}], schoolIds:[],
personIds:[], medicalMemberIds:[] }
```

Response

```
201 { ...mission with nested relations }
```

PATCH `/api/missions/:id`

Edit status/dates/summary/counts; syncs logs + availability.

Request

```
{ status?, startDate?, endDate?, summary?, numPeople?, ... }
```

Response

```
200 { ...mission }
```

DELETE `/api/missions/:id`

Delete mission + generated logs; frees members.

Response

```
204
```

Database tables used

Mission, MissionCountry, MissionSchool, MissionPerson, MissionMedical, MissionLog, TaskRecord

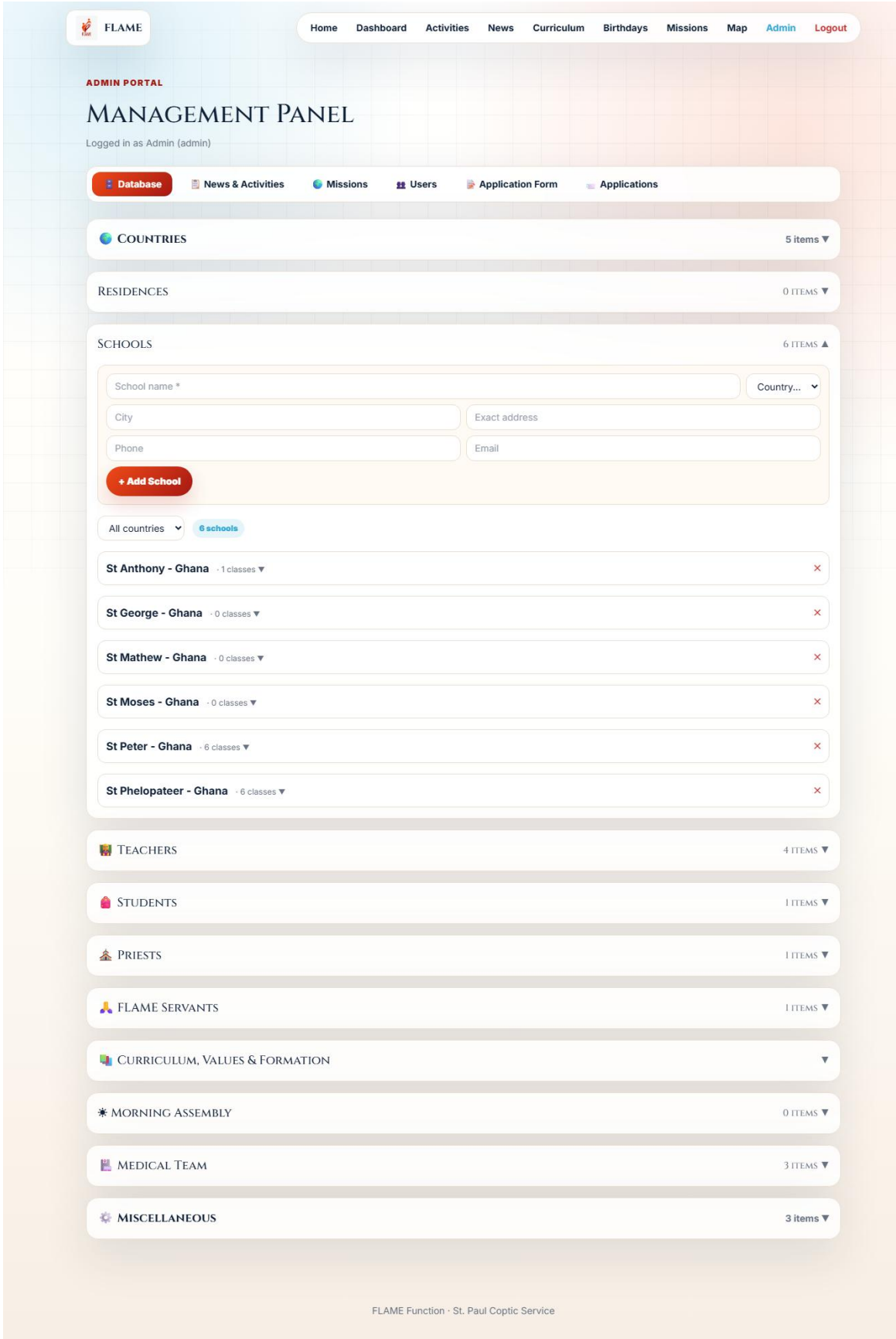
2.4 School Task Board (Logs, Visits, Content)

Per-school hub: create medical logs / principal visits / tasks (summary + date mandatory), assign people with roles, and track assigned curriculum/values/formation with a status.

How to use it

17. Admin → Database → Schools → expand a school.
18. 'Add Log / Task': choose a type, set date, write a summary (both required), pick a status.
19. For a Values Class, select which values class from the extra selector that appears.

20. 'People & Roles': assign an existing person as Principal / Worker / Resident / Teacher / Priest, change a role, or remove from the school.
21. Update any log or assigned-content status (Expected / Pending / Completed) inline.
22. Manage Classes and add Students (linked to a class) in the same panel.


FLAME
Home
Dashboard
Activities
News
Curriculum
Birthdays
Missions
Map
Admin
Logout
ADMIN PORTAL
MANAGEMENT PANEL
Logged in as Admin (admin)
Database
News & Activities
Missions
Users
Application Form
Applications
COUNTRIES
5 items
RESIDENCES
0 ITEMS
SCHOOLS
6 ITEMS
School name *
Country...
City
Exact address
Phone
Email
+ Add School
All countries
6 schools
St Anthony - Ghana · 1 classes
St George - Ghana · 0 classes
St Mathew - Ghana · 0 classes
St Moses - Ghana · 0 classes
St Peter - Ghana · 6 classes
St Phelopateer - Ghana · 6 classes
TEACHERS
4 ITEMS
STUDENTS
1 ITEMS
PRIESTS
1 ITEMS
FLAME SERVANTS
1 ITEMS
CURRICULUM, VALUES & FORMATION
MORNING ASSEMBLY
0 ITEMS
MEDICAL TEAM
3 ITEMS
MISCELLANEOUS
3 items
FLAME Function · St. Paul Coptic Service

Business rules

- School-level logs require a non-empty summary and a date (HTTP 400 otherwise). The summary is stored in MissionLog.notes and mirrored as a TaskRecord 'Summary: ...'.
- Logging a Values Class also assigns it to the school (if not already) and syncs the per-school status.
- Assigning a person sets Person.schoolId and Person.position; removing clears schoolId.
- GET /schools/:id/tasks aggregates MissionLogs + CurriculumSchool + ValuesClassSchool + FormationSchool with their statuses.

Frontend

components/admin/SchoolManager.jsx (SchoolDetail): task board, Add Log/Task form with conditional values-class selector, People & Roles, Classes + StudentForm.

API

GET `/api/schools/:id/tasks`

Everything happening at a school, with statuses.

Response

```
{ logs:[{id,title,logDate,status,notes}], curriculum:[{id,title,status}],
values:[...], formation:[...] }
```

POST `/api/logs`

Create a school-level log/task.

Request

```
{ schoolId, logTypeId, summary, logDate, status, responsiblePersonId? }
```

Response

```
201 { ...log }
```

PATCH `/api/logs/:id`

Edit a log (summary/date/status/type).

Request

```
{ summary?, logDate?, status?, logTypeId? }
```

Response

```
200 { ...log }
```

GET `/api/logs/types/school`

Creatable school log types.

Response

```
[ { id, label, icon } ]
```

Database tables used

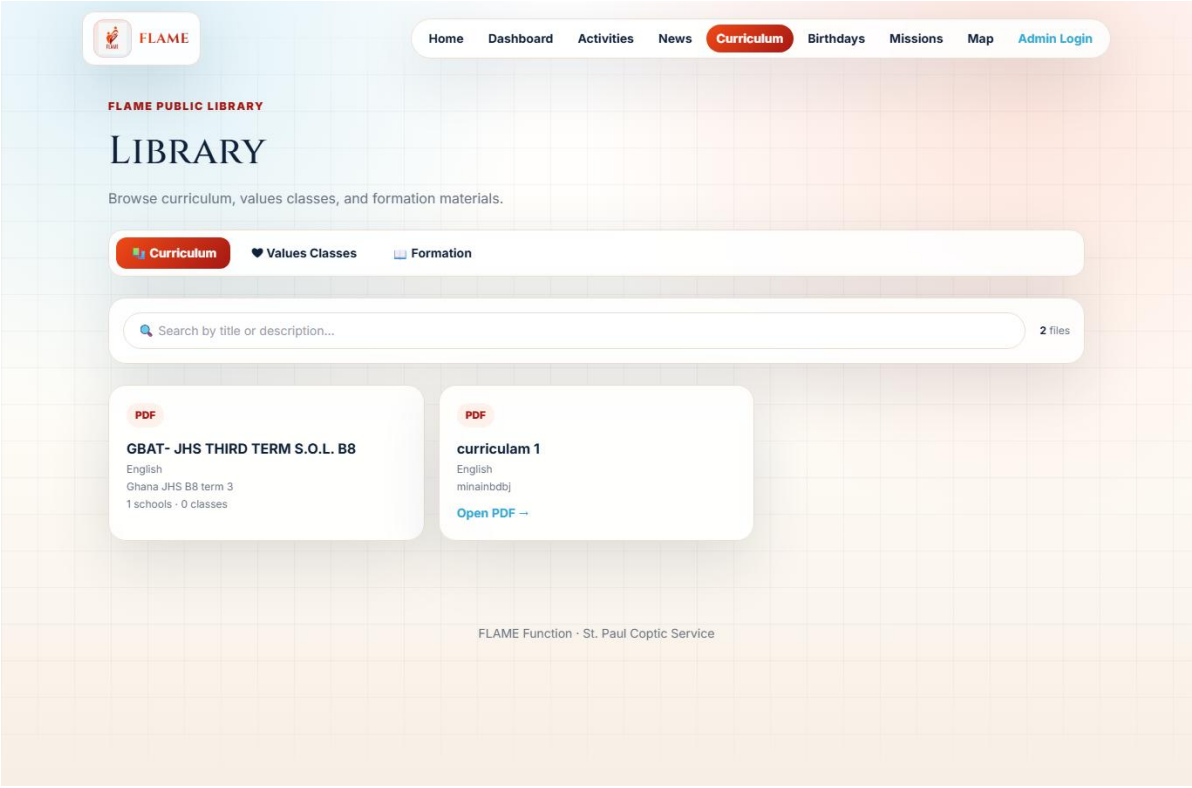
School, MissionLog, TaskRecord, Person, ClassLevel, Student, CurriculumSchool, ValuesClassSchool, FormationSchool

2.5 Curriculum, Values & Formation

Upload PDF documents (≤12MB) and assign them to multiple schools and classes (classes scoped to the chosen schools). Public visitors browse them by tab and open the PDFs.

How to use it

23. Admin → Database → Curriculum, Values & Formation.
24. Enter a title + language and upload a PDF.
25. Select schools; the class picker then only lists classes in those schools.
26. Use 'Edit assignments' on an existing item to re-assign schools/classes later.
27. Public users open /curriculum, switch tabs, and open the PDFs.



Business rules

- Uploads are limited to 12MB (PDF) and stored on disk; the public URL is /uploads/<uuid>.pdf (proxied in dev, served statically in prod).
- A school assignment carries a status (expected|pending|completed) updatable per school.

- Deleting a document cascades to its class/school join rows.

Frontend

components/admin/DocumentsAdmin.jsx (reusable DocManager used for all three) + public pages/public/Curriculum.jsx (Curriculum / Values / Formation tabs). Uploads via multipart/form-data.

API

GET `/api/curriculum-docs`

List with classes + schools.

Response

```
[ { id, title, language, fileUrl, classes:[...],
schools:[{schoolId,status,school}] } ]
```

POST `/api/curriculum-docs`

Upload + assign (multipart).

Request

```
form-data: title, description, language, file, schoolIds(JSON), classIds(JSON)
```

Response

```
201 { ...curriculum }
```

PATCH `/api/curriculum-docs/:id`

Edit meta + re-assign schools/classes.

Request

```
{ title?, schoolIds?, classIds? }
```

Response

```
200 { ...curriculum }
```

PATCH `/api/curriculum-docs/:id/school/:schoolId`

Set per-school status.

Request

```
{ status }
```

Response

```
200 { ...join }
```

Database tables used

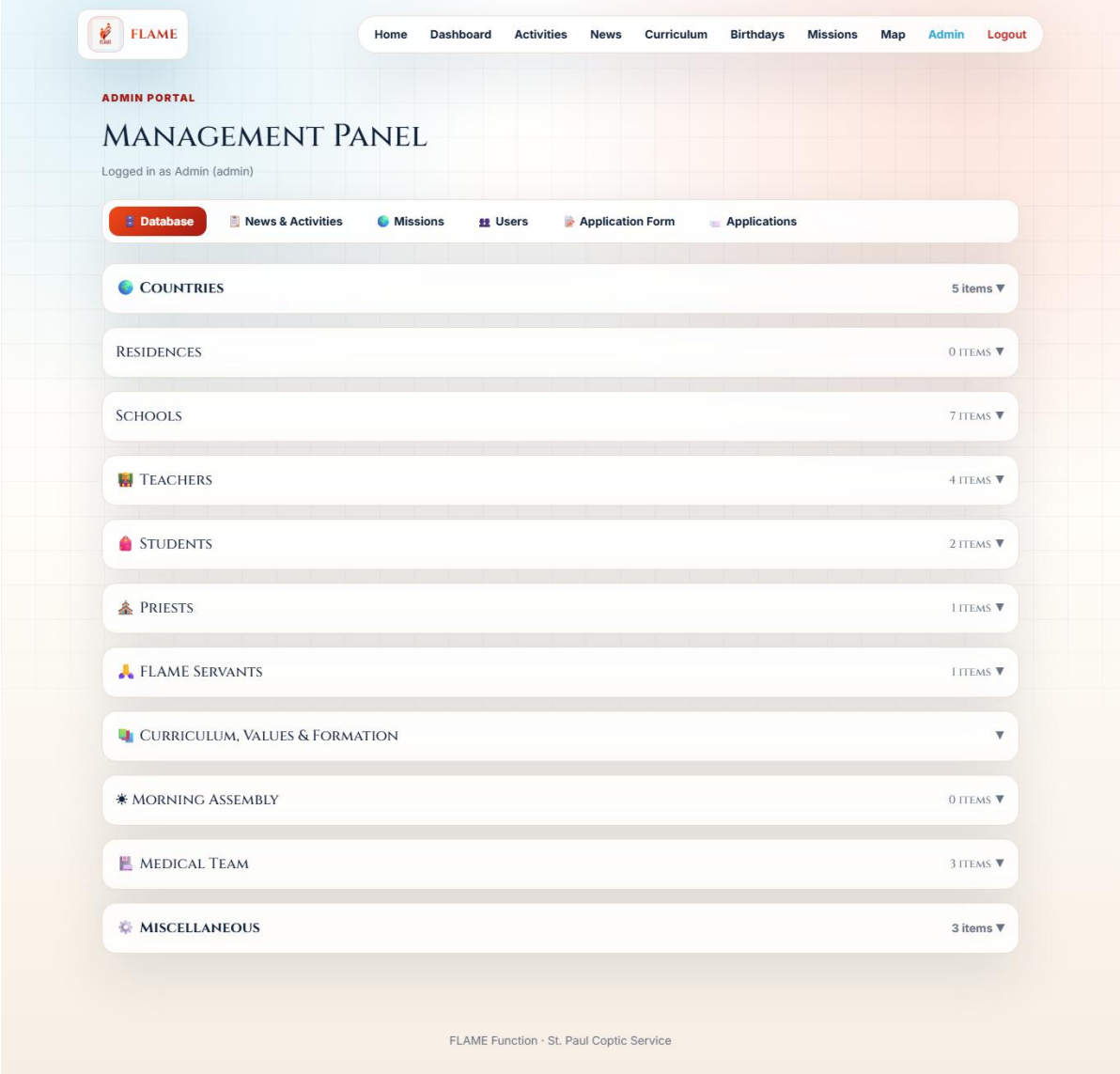
Curriculum, CurriculumClass, CurriculumSchool, ValuesClass, ValuesClassClass, ValuesClassSchool, Formation, FormationSchool

2.6 👥 People: Teachers, Students, Priests, Servants

A single reusable Person model powers teachers, priests, servants and school staff; Students are separate and belong to a class. Birthdate cannot be in the future. Country/school filters with live counts and full inline editing.

How to use it

28. Admin → Database → Teachers / Students / Priests / Servants.
29. Create a record (name required; students need a school + class).
30. Filter by country and school; the section header shows the live count.
31. New records appear instantly — filters auto-clear on create.
32. Use Edit on any row to update details; birthdate pickers block future dates.



Business rules

- Person.fullName is the only required field; a Person is reusable across schools, residences and missions.
- Server rejects future birthdate/dob (HTTP 400).
- People filter: ?type=<position>, ?isPriest=true, ?schoolId=, ?countryId= (matches school's country OR priest's assigned country).
- Student must link to a class (and therefore a school).
- Person.available drives the busy/free marker used in mission selection.

Frontend

TeachersAdmin.jsx, StudentsAdmin.jsx, PriestsAdmin.jsx, ServantsAdmin.jsx, PersonPicker.jsx. Counts rendered via Collapsible countEndpoint.

API

GET /api/people

List/filter people.

Request

```
query: type?, isPriest?, schoolId?, countryId?, available?
```

Response

```
[ { id, fullName, birthdate, position, isPriest, available, school,
priestCountry, residence, classesTaught } ]
```

POST /api/people

Create a person.

Request

```
{ fullName, position?, birthdate?, schoolId?, priestCountryId?, isPriest? }
```

Response

```
201 { ...person }
```

PATCH /api/people/:id

Edit / assign to school / change role.

Request

```
{ schoolId?, position?, available?, ... }
```

Response

```
200 { ...person }
```

GET/POST/PATCH/DELETE /api/students

Student CRUD.

Request

```
{ fullName, classId, birthday?, phone?, fatherName?, ... }
```

Response

```
{ ...student }
```

Database tables used

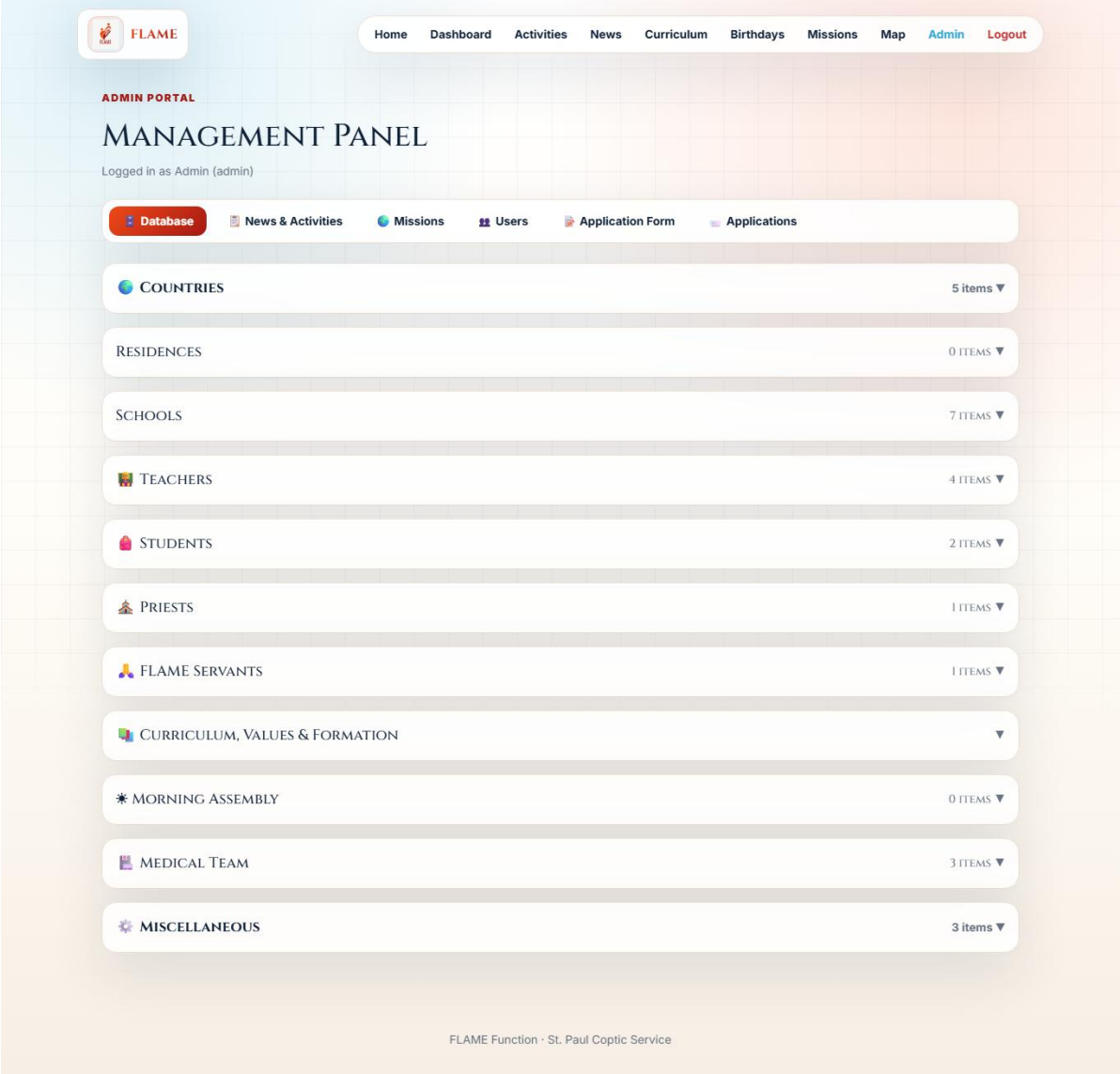
Person, Student, ClassLevel, School, Country, Residence

2.7 Medical Team & Availability

Medical team members with specialty and availability. Availability is surfaced when selecting members for medical missions and flips automatically while a mission is in progress.

How to use it

33. Admin → Database → Medical Team.
34. Add a member (name, specialty, contact, email).
35. Toggle availability or Edit the member inline.
36. When creating a Medical mission, members show a green/red availability marker.



Business rules

- A Medical mission set to Pending marks its selected medics unavailable; Completed/Delete frees them.
- MissionMedical is a composite-key join (missionId + memberId).

Frontend

components/admin/MedicalTeamAdmin.jsx (availability toggle + inline edit); consumed by MissionsAdmin.jsx.

API

GET `/api/medical-team`

List members.

Response

```
[ { id, fullName, specialty, contact, email, available } ]
```

POST `/api/medical-team`

Add a member.

Request

```
{ fullName, specialty?, contact?, email? }
```

Response

```
201 { ...member }
```

PATCH `/api/medical-team/:id`

Edit / toggle availability.

Request

```
{ available? , ... }
```

Response

```
200 { ...member }
```

Database tables used

MedicalTeamMember, MissionMedical, Mission

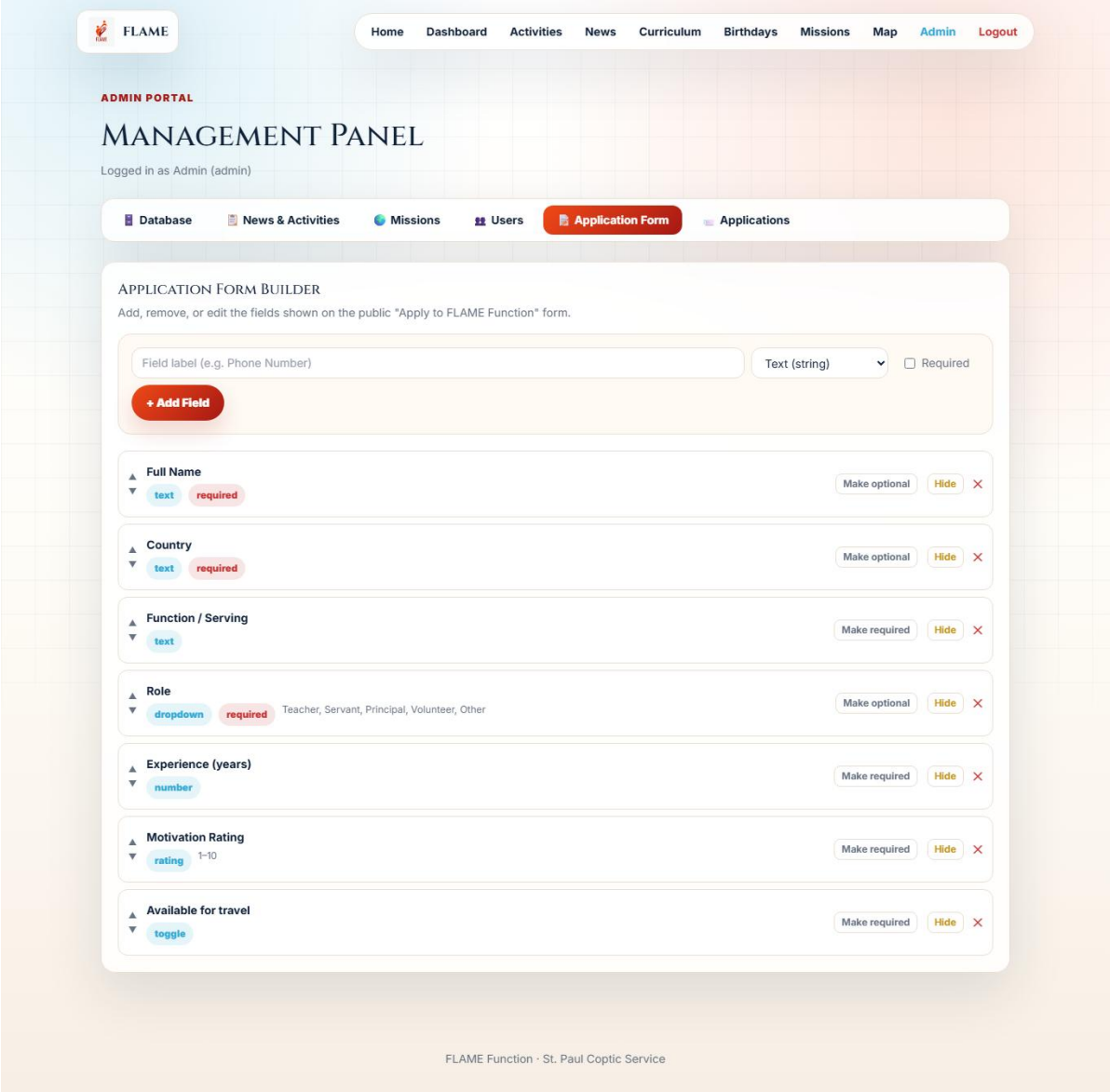
2.8 Dynamic Application Form

Admins define the Apply form fields (text / number / dropdown / toggle / rating 1–10). Visitors submit via a popup; admins review all submissions.

How to use it

37. Admin → Application Form.
38. Add fields and pick a type; for dropdown provide the option values.
39. Mark required, set order, or remove fields.
40. Visitors click Apply on Home and submit.

41. Admin → Applications to review every submission.



Business rules

- FormField.key is unique; type ∈ {text, number, dropdown, toggle, rating}; dropdown options stored in `options`.
- Only active fields render on the public form, ordered by `order`.
- Application.values is a JSON string of answers keyed by field key.

Frontend

components/admin/FormBuilder.jsx, components/ApplyModal.jsx, components/admin/ApplicationsAdmin.jsx.

API

GET /api/form-fields

List fields.

Response

```
[ { id, label, key, type, options, required, order, active } ]
```

POST/PATCH/DELETE /api/form-fields

Manage fields.

Request

```
{ label, key, type, options?, required?, order? }
```

Response

```
{ ...field }
```

GET /api/applications

List submissions (reviewer/admin).

Response

```
[ { id, values, submittedAt } ]
```

POST /api/applications

Submit (public).

Request

```
{ values:{...} }
```

Response

```
201 { id, submittedAt }
```

Database tables used

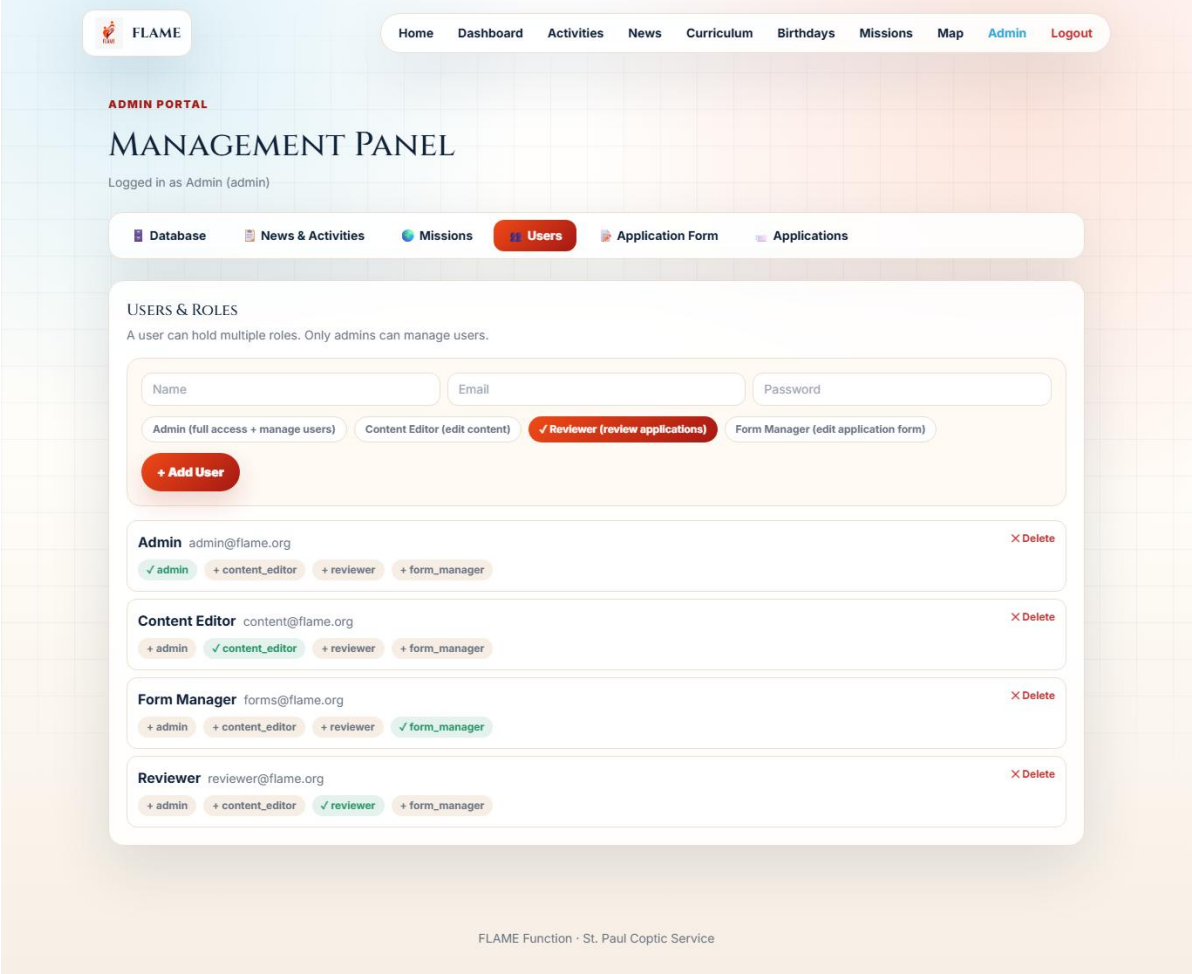
FormField, Application

2.9 Users, Roles & Permissions

Multi-role access control via JWT. Admin is the only role that manages users. Roles: admin, content_editor, reviewer, form_manager.

How to use it

42. Log in as admin (POST /api/auth/login returns a JWT).
43. Admin → Users.
44. Create a user and assign one or more roles (stored comma-separated).
45. Each role grants scoped access, enforced by requireRole on the server.



Business rules

- CAN_EDIT_CONTENT = admin, content_editor. CAN_REVIEW_APPLICATIONS = admin, reviewer. CAN_MANAGE_FORM = admin, form_manager. CAN_MANAGE_USERS = admin only.
- requireRole grants access if ANY of the user's roles is permitted.
- The JWT carries the roles array; the client attaches it as Authorization: Bearer <token>.
- Seed users: admin@flame.org / Flame@Admin2026 (and content@, reviewer@, forms@).

Frontend

components/admin/UsersAdmin.jsx, auth context lib/auth.jsx, components/ProtectedRoute.jsx, axios interceptor in api/client.js.

API

POST `/api/auth/login`

Authenticate.

Request

```
{ email, password }
```

Response

```
200 { token, user:{ id, name, roles } }
```

GET/POST/PATCH/DELETE `/api/users`

Manage users (admin only).

Request

```
{ name, email, password, role }
```

Response

```
{ ...user }
```

Database tables used

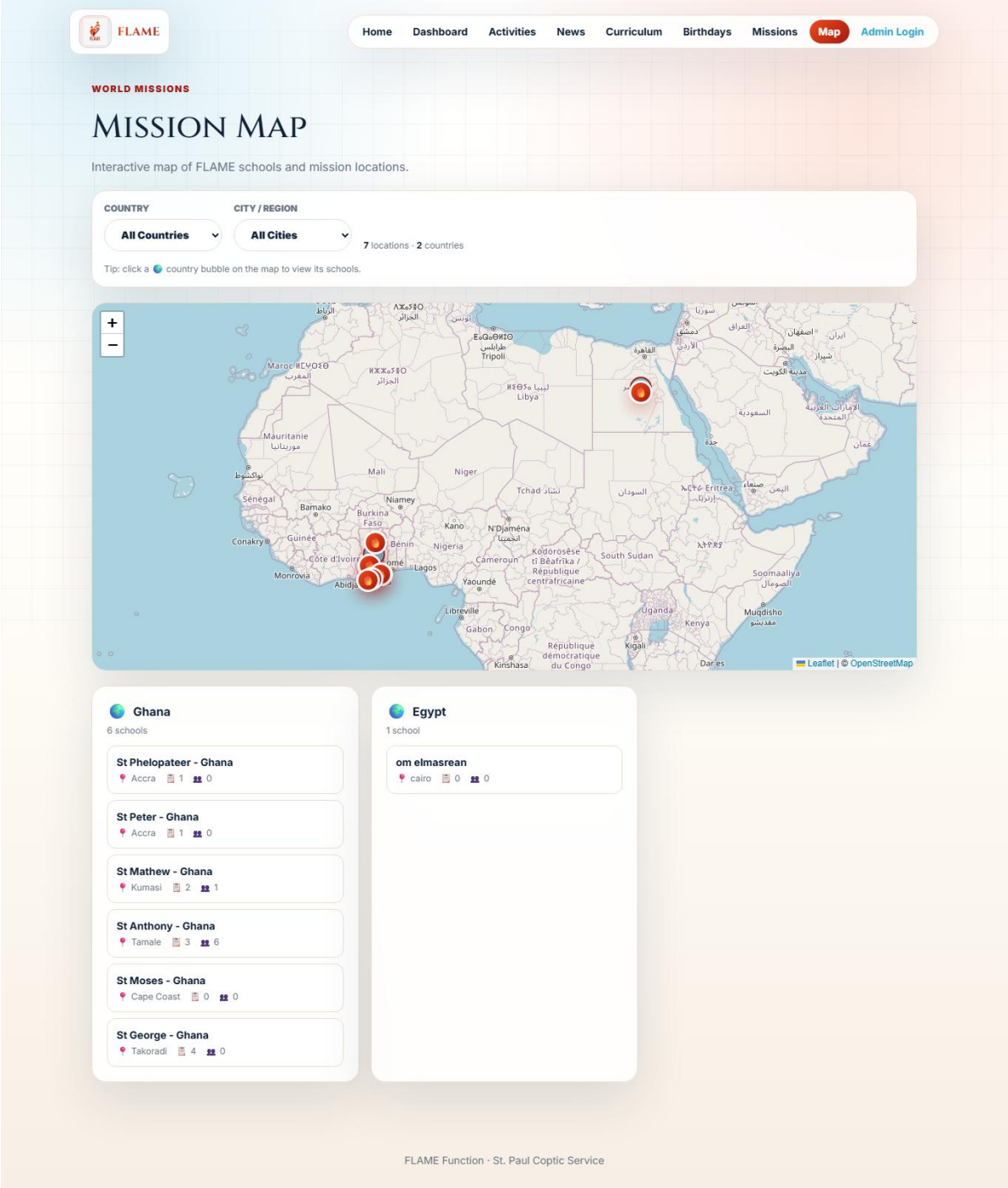
User

2.10 🗺 World Mission Map

Leaflet map of school locations with mission and people counts, filterable by country and city — filtering handled server-side.

How to use it

46. Open /map.
47. Browse pins for each geolocated school (lat/lng) with mission + people counts.
48. Filter by country or city.



Business rules

- Only schools with latitude and longitude set appear as pins.
- Counts come from Prisma _count on missionLogs and people; results are grouped by country for the sidebar.

Frontend

pages/public/WorldMap.jsx using react-leaflet@4 (React 18 compatible).

API

GET `/api/map/locations`

Geolocated schools + counts.

Request

```
query: country?, city?
```

Response

```
{ locations:[{id,name,country,city,lat,lng,missions,people}], byCountry, total }
```

GET `/api/map/filters`

Distinct countries + cities.

Response

```
{ countries:[...], cities:[...] }
```

Database tables used

School, MissionLog

2.11 ✨ Platform UX: Friendly Date Picker & Brand

A custom, responsive calendar date picker replaces the native input everywhere, and the FLAME logo has an enhanced animated brand treatment.

How to use it

49. Click any date field (dashboard range, mission dates, birthdays, logs).
50. A calendar popover opens with month/year quick-select and ‹ › navigation.
51. Pick a day, or use Today / Clear; disabled days respect min/max.
52. On mobile the picker becomes a centered sheet with a dimmed backdrop.

Business rules

- The picker renders with fixed positioning so it never gets clipped by card overflow, flips above when space is tight, and clamps to the viewport.
- Values use the ISO YYYY-MM-DD format, so it is a drop-in replacement for the native date input across all forms.
- Animations respect prefers-reduced-motion.

Frontend

components/DateField.jsx (self-contained, no external library) used by Dashboard, MissionsAdmin, SchoolManager, Teachers/Students, PersonPicker, Morning Assembly. Logo treatment via .flame-logo-hero / .flame-logo-nav in index.css.

API

– `(client-only)`

No API; pure UI component and styling.

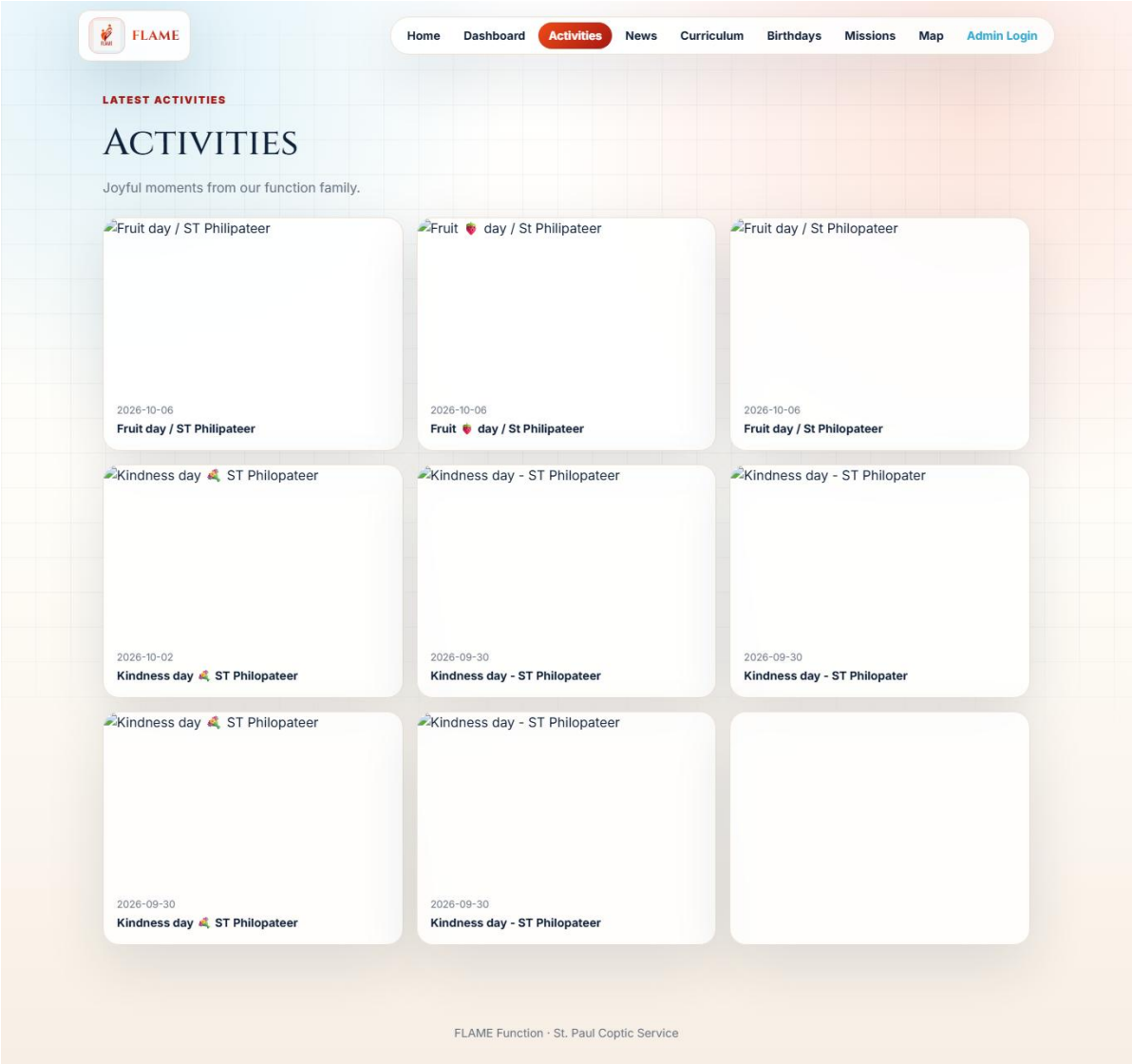
Database tables used

2.12 🎉 Activities, News & Birthdays

Admin-managed content surfaced to the public: activities (school / local_flame / other), news posts, and a birthday calendar aggregating people and students — each birthday showing a copyable contact number.

How to use it

53. Admin manages Activities and News under News & Activities.
54. Birthdays are computed from Person.birthdate (or legacy dob) and Student.birthday.
55. Public users open /activities, /news, /birthdays.
56. In Birthdays, click the 📞 contact chip to copy the number to the clipboard.



Business rules

- Activity.scope ∈ {school, local_flame, other}; may link to a school.
- Birthdays endpoint merges Person and Student records and sorts by month/day.
- Each birthday carries a contact: a person's contact, or a student's phone / parent contact (labelled).

Frontend

pages/public/Activities.jsx, News.jsx, Birthdays.jsx; admin CrudSections + news admin.

API

GET/POST `/api/activities`

Activities feed.

Request

```
{ caption, activityDate, scope, summary, schoolId? }
```

Response

```
{ ...activity }
```

GET/POST `/api/news`

News posts.

Request

```
{ title, body, postDate }
```

Response

```
{ ...post }
```

GET `/api/birthdays`

Merged birthday calendar.

Response

```
[ { name, date, source } ]
```

Database tables used

Activity, NewsPost, Person, Student

3. Database Reference

Every table with typed columns. Key = PK (primary key) / FK→Target (foreign key). '?' marks a nullable column.

User

Admin/staff accounts. Roles stored comma-separated.

Column	Type	Key / Constraint	Notes
id	String	PK	uuid
name	String		
email	String	UNIQUE	
passwordHash	String		bcrypt
role	String		CSV of roles, default 'reviewer'
createdAt	DateTime		now()

Country

Top of the geography tree.

Column	Type	Key / Constraint	Notes
id	String	PK	uuid
name	String	UNIQUE	
code	String?		e.g. GH
active	Boolean		default true
createdAt	DateTime		

Residence

A residence belongs to a country; houses people.

Column	Type	Key / Constraint	Notes
id	String	PK	
name	String		
city	String?		
address	String?		
countryId	String?	FK→Country	
createdAt	DateTime		

School

A school in a country; owns classes, staff, logs, content.

Column	Type	Key / Constraint	Notes

id	String	PK	
name	String		
city	String?		
address	String?		
phone	String?		
email	String?		
countryId	String?	FK→Country	
region	String?		holds city for map
workingDays	String		default '1,2,3,4,5'
latitude	Float?		
longitude	Float?		
active	Boolean		default true

Person

Central reusable person: teacher/priest/worker/resident/servant.

Column	Type	Key / Constraint	Notes
id	String	PK	
fullName	String	REQUIRED	only mandatory field
birthdate	DateTime?		no future dates
nationality	String?		
position	String?		Principal|Teacher|Worker|Resident|Priest|Servant|Other
contact	String?		
email	String?		
isPriest	Boolean		default false
available	Boolean		default true
dob	DateTime?		legacy birthday
schoolId	String?	FK→School	
residenceId	String?	FK→Residence	
priestCountryId	String?	FK→Country	

ClassLevel

A class within a school with one teacher.

Column	Type	Key / Constraint	Notes
id	String	PK	
name	String		
stage	String?		
order	Int		default 0
schoolId	String?	FK→School	
teacherId	String?	FK→Person	
active	Boolean		default true

Student

A student belonging to a class (and thus a school).

Column	Type	Key / Constraint	Notes
id	String	PK	
fullName	String	REQUIRED	
birthday	DateTime?		no future
phone	String?		
fatherName	String?		
fatherContact	String?		
motherName	String?		
motherContact	String?		
classId	String?	FK→ClassLevel	

MedicalTeamMember

Selectable in medical missions; has availability.

Column	Type	Key / Constraint	Notes
id	String	PK	
fullName	String	REQUIRED	
specialty	String?		
contact	String?		
email	String?		
available	Boolean		default true

Curriculum

PDF curriculum (≤12MB), multi-assignable.

Column	Type	Key / Constraint	Notes
id	String	PK	
title	String		
description	String?		
language	String?		ar|en|fr
fileUrl	String?		/uploads/..pdf

CurriculumClass

Curriculum ↔ Class (M:N).

Column	Type	Key / Constraint	Notes
curriculumId	String	PK/FK→Curriculum	
classId	String	PK/FK→ClassLevel	

CurriculumSchool

Curriculum ↔ School with per-school status.

Column	Type	Key / Constraint	Notes
curriculumId	String	PK/FK→Curriculum	
schoolId	String	PK/FK→School	
status	String		expected|pending|completed

ValuesClass

Values-class PDF (same model as curriculum).

Column	Type	Key / Constraint	Notes
id	String	PK	
title	String		
description	String?		
language	String?		
fileUrl	String?		

ValuesClassClass

ValuesClass ↔ Class (M:N).

Column	Type	Key / Constraint	Notes
valuesClassId	String	PK/FK	
classId	String	PK/FK→ClassLevel	

ValuesClassSchool

ValuesClass ↔ School with status.

Column	Type	Key / Constraint	Notes
valuesClassId	String	PK/FK	
schoolId	String	PK/FK→School	
status	String		expected|pending|completed

Formation

Formation files, multi-school.

Column	Type	Key / Constraint	Notes
id	String	PK	

title	String		
description	String?		
fileUrl	String?		

FormationSchool

Formation ↔ School with status.

Column	Type	Key / Constraint	Notes
formationId	String	PK/FK	
schoolId	String	PK/FK→School	
status	String		expected|pending|completed

Mission

A field mission; generates dashboard logs.

Column	Type	Key / Constraint	Notes
id	String	PK	
title	String		
type	String		Medical|Follow-ups|..
status	String		expected|pending|completed
startDate	DateTime?		required by API
endDate	DateTime?		required by API, ≥ start
summary	String?		
numPeople	Int?		
durationDays	Int?		
anonymousCount	Int		default 0

MissionCountry

Mission ↔ Country with per-country dates.

Column	Type	Key / Constraint	Notes
id	String	PK	
missionId	String	FK→Mission (cascade)	
countryId	String	FK→Country	
startDate	DateTime?		
endDate	DateTime?		

MissionSchool

Mission ↔ School visited (composite PK).

Column	Type	Key / Constraint	Notes
missionId	String	PK/FK (cascade)	
schoolId	String	PK/FK→School	

MissionPerson

Mission ↔ Person (team).

Column	Type	Key / Constraint	Notes
missionId	String	PK/FK (cascade)	
personId	String	PK/FK→Person	

MissionMedical

Mission ↔ MedicalTeamMember.

Column	Type	Key / Constraint	Notes
missionId	String	PK/FK (cascade)	
memberId	String	PK/FK→MedicalTeamMember	

MorningAssembly

Six sections, each text OR an uploaded file.

Column	Type	Key / Constraint	Notes
id	String	PK	
schoolId	String?	FK→School	
date	DateTime?		
openingPrayer(+File)	String?		
physicalExercise(+File)	String?		
radio(+File)	String?		
nationalAnthem(+File)	String?		
inspection(+File)	String?		
announcement(+File)	String?		

LogType

Dashboard category for logs.

Column	Type	Key / Constraint	Notes
id	String	PK	e.g. medical_log
key	String	UNIQUE	
label	String		
icon	String?		
expectedPerWorkingDay	Int		default 0

MissionLog

The dashboard record; unifies missions and school tasks.

Column	Type	Key / Constraint	Notes
id	String	PK	
logTypeId	String	FK→LogType	
schoolId	String	FK→School	
responsiblePersonId	String?	FK→Person	
logDate	DateTime		
status	String		expected|pending|completed
notes	String?		holds 'mission:<id>' tag or summary
createdById	String?	FK→User	

TaskRecord

Granular performed/not-performed items under a log.

Column	Type	Key / Constraint	Notes
id	String	PK	
missionLogId	String	FK→MissionLog (cascade)	
question	String		
performed	Boolean		default false
responsible	String?		
recordedDate	DateTime?		

Activity

Public activity feed item.

Column	Type	Key / Constraint	Notes
id	String	PK	
caption	String?		
activityDate	DateTime?		
scope	String		school|local_flame|other
summary	String?		
schoolId	String?	FK→School	
imageUrl	String?		

NewsPost

Public news post.

Column	Type	Key / Constraint	Notes
id	String	PK	
title	String		
body	String?		
postDate	DateTime?		
imageUrl	String?		

FormField

A field in the dynamic Apply form.

Column	Type	Key / Constraint	Notes
id	String	PK	
label	String		
key	String	UNIQUE	
type	String		text|number|dropdown|toggle|rating
options	String?		dropdown values
required	Boolean		default false
order	Int		default 0
active	Boolean		default true

Application

A submitted Apply form.

Column	Type	Key / Constraint	Notes
id	String	PK	
values	String		JSON of answers
submittedAt	DateTime		

MiscEntry

Generic reference/lookup entries.

Column	Type	Key / Constraint	Notes
id	String	PK	
category	String		
label	String		
value	String?		
notes	String?		

4. Entity-Relationship Diagram

Tables grouped by domain (Geography & People · Content · Missions & Logs · Feed & Admin) with their relationships. PK in red, FK in blue.

